

Validating an LLM Judge: Kappa and Position Bias, by Hand

Why a judge is a number generator until you measure κ against humans — and why a verdict that flips when you swap the order is measuring position, not quality.

What this document does. It takes the one validation table from Module 3.6 — the *Trajectory Judge* scored against a human gold set of 200 refund-resolution trajectories — and rebuilds its agreement number from the raw confusion-matrix cells by hand. Then it takes a small set of 20 *pairwise* A-vs-B comparisons, runs each one in both orders, and computes exactly how much the win-rate swings purely because of which option was listed first. Nothing is asserted. Every number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The module quotes the judge at $\kappa = 0.68$ on 200 trajectories; we rebuild that figure from its four cells, then show the position-bias correction that has to run *before* any pairwise number is trusted.

Where this sits. This is **Topic 1 of Module 3.6** — the math behind the “validate before you quote” rule and the position-swap mitigation pattern. Its companions: Module 3.5 (building the human gold set the judge is measured against) and Topic 2 of this module (jury voting, where averaging several judges cuts the leftover noise by $1/\sqrt{n}$).

Concepts covered, in order: the 2×2 confusion matrix · observed agreement P_o · chance agreement P_e from the marginals · Cohen’s $\kappa = (P_o - P_e)/(1 - P_e)$ · the $\kappa \geq 0.65$ deploy floor · position bias as a verdict flip under order swap · positional win-rate swing · the debiased swap-and-average win-rate.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one judge, one gold set, small enough to count by hand

A judge that you have not validated is, in the module’s words, “a number generator.” To turn its verdicts into trustworthy numbers you do two separate things. **(1)** You measure how often it agrees with humans — but corrected for the agreement you would get by pure luck — using Cohen’s κ . **(2)** For *pairwise* judges (A vs B) you measure how much of the verdict is driven by the *order* you listed the options in, not their content. This document works both, by hand, on tiny grounded sets.

The quantities. We label every trajectory in the gold set `{pass, fail}` twice: once by a human panel (the gold), once by the judge. That gives a 2×2 table. For the pairwise part, every pair is judged twice — once with option A shown first, once with B shown first — and we record which content the judge named the winner.

Quantity	Symbol	Value here
Trajectories in the gold set	N	200
Both say pass	n_{11}	150
Judge pass , human fail (false pass)	n_{12}	8
Judge fail , human pass (false fail)	n_{21}	12
Both say fail	n_{22}	30
Deploy floor on κ	—	0.65
Pairwise comparisons (A vs B)	m	20
Price ratio: judge vs human label	—	$\sim 1/50$

Notation. P_o is *observed* agreement (fraction of trajectories the judge and humans label the same). P_e is *expected* agreement under chance, built from the row and column totals. κ rescales P_o so that 0 means “no better than guessing” and 1 means “perfect.” We carry four decimals throughout.

Intuition

Raw agreement lies. If 90% of trajectories truly **pass**, a judge that stamps **pass** on everything agrees with humans 90% of the time while knowing nothing. κ subtracts off exactly that free lunch: it asks “how much agreement is left *after* we account for the agreement chance alone would have produced?” A high P_o with a low κ is the signature of a judge that has just learned the base rate.

2 Step 1 — the confusion matrix and raw agreement

Lay the 200 trajectories into the 2×2 table. The diagonal is agreement; the off-diagonal is the two ways the judge can be wrong.

	Human pass	Human fail	Judge total
Judge pass	$n_{11} = 150$	$n_{12} = 8$	158
Judge fail	$n_{21} = 12$	$n_{22} = 30$	42
Human total	162	38	200

Observed agreement is the diagonal over the total:

$$P_o = \frac{n_{11} + n_{22}}{N} = \frac{150 + 30}{200} = \frac{180}{200} = 0.90.$$

So the judge and the humans land on the same verdict 90% of the time, and *disagree* 10% of the time. ✓ That is the module’s “raw agreement” cell. But we are not done — 0.90 is inflated by the fact that **pass** is common.

3 Step 2 — chance agreement from the marginals

How often would the judge and humans agree if each one just rolled dice with their own observed rates? The judge says **pass** on $158/200 = 0.79$ of cases; the humans on $162/200 = 0.81$. Two

independent rollers both land on **pass** with probability 0.79×0.81 , and both land on **fail** with probability 0.21×0.19 . Add the two ways to agree:

$$P_e = \underbrace{\frac{n_{11}+n_{12}}{N} \cdot \frac{n_{11}+n_{21}}{N}}_{\text{both pass}} + \underbrace{\frac{n_{21}+n_{22}}{N} \cdot \frac{n_{12}+n_{22}}{N}}_{\text{both fail}} \\ = (0.79)(0.81) + (0.21)(0.19) = 0.6399 + 0.0399 = 0.6798. \quad (1)$$

So chance alone — two parties matched only on their base rates — already produces $\approx 68\%$ agreement. ✓ That is the bar P_o has to clear to mean anything.

Mini-example — this operation in isolation

The marginals are the only inputs to P_e ; the diagonal never enters. Judge-**pass** rate = $\frac{150+8}{200} = 0.79$. Human-**pass** rate = $\frac{150+12}{200} = 0.81$. Product = 0.6399. Repeat for **fail**: $0.21 \times 0.19 = 0.0399$. Sum = 0.6798. If the judge stamped **pass** on *everything*, its **pass** marginal would be 1.0, and $P_e = 1 \cdot 0.81 + 0 = 0.81$ — almost exactly its P_o , which is the tell that κ will collapse to near zero.

4 Step 3 — Cohen’s κ : agreement above chance

κ is the agreement that survives once chance is removed, normalised by the most that *could* have survived $(1 - P_e)$:

$$\kappa = \frac{P_o - P_e}{1 - P_e} \quad (2)$$

Substituting Equations (1) and the P_o from Step 1:

$$\kappa = \frac{0.90 - 0.6798}{1 - 0.6798} = \frac{0.2202}{0.3202} = 0.6877 \approx 0.68.$$

✓ This is the module’s quoted $\kappa = 0.68$ — “substantial” agreement, above the deploy floor. Notice how much smaller it is than the raw 0.90: of the 90% agreement, 68 points were free, and only the remaining 0.2202 out of a possible 0.3202 is real skill — which is exactly 0.69 of the way there.

Intuition

κ answers “what fraction of the *non-trivial* agreement did the judge actually capture?” The numerator $P_o - P_e = 0.2202$ is how far above chance it landed; the denominator $1 - P_e = 0.3202$ is how far above chance a *perfect* judge could land. Their ratio is the score. A judge that only matched the base rate would have $P_o = P_e$, numerator 0, and $\kappa = 0$ — correctly flagged as worthless no matter how high its raw agreement looked.

5 Step 4 — the $\kappa \geq 0.65$ deploy floor

Why gate deployment at $\kappa \geq 0.65$ and not just “high raw agreement”? Because κ ties directly to how often the judge *actually* disagrees with humans on the cases that matter. Compare our judge to a noisier one run on the same $N = 200$:

Judge	P_o	P_e	κ	disagree	verdict
Deployed (this doc)	0.90	0.6798	0.688	10%	ship
Below floor	0.75	0.6398	0.306	25%	do not ship

The below-floor judge still agrees with humans three times out of four — it “feels” fine — but its κ of 0.31 says only a third of its non-trivial agreement is real, and it disagrees with humans on a full quarter of trajectories. A judge below $\kappa \approx 0.65$ is wrong, relative to your gold humans, on *more than a third* of the cases where the answer is not obvious. Every score it then emits inherits that error. The floor is not a ritual; it is the line below which the judge’s disagreement with humans is too large to quote.

Watch out

κ is computed on *one* collapsed binary verdict (**pass/fail**). It does *not* certify the judge’s 1–5 Likert axes — those need a separate ordinal agreement measure (ICC, or quadratic-weighted κ). And κ inherits the gold set’s blind spots: if your human panel never labelled a certain failure mode, a high κ proves nothing about it. Validate on a gold set that is stratified across the categories you actually ship.

6 Step 5 — position bias: the verdict that flips on order

Cohen’s κ validated a *pointwise* judge. **Pairwise** judges (“is trajectory A or B better?”) carry a second disease: they tend to favour whichever option is listed *first*, regardless of content. The test is brutal and simple. Show every pair twice:

pass 1: (A, B) pass 2: (B, A),

changing *only* the order. A pair whose named winner **flips** between the two passes was decided by position, not quality. Over our $m = 20$ pairs:

Outcome over both passes	count	share
Consistent: same content wins in both orders	15	75%
Flipped: winner changed when only order changed	5	25%
Total pairs	20	100%

The **position-bias rate** is the flip fraction, $5/20 = 25\%$. One pair in four was decided by a coin the judge did not know it was flipping. Now read the same data as a win-rate for content A, split by where A sat:

Where A was listed	A wins	of	A win-rate
Position 1 (shown first)	14	20	70%
Position 2 (shown second)	9	20	45%
Positional swing	$70\% - 45\% = 25 \text{ pts}$		

The *same* content A wins 70% of the time when it sits first and only 45% when it sits second — a **25-point positional swing** from nothing but order. ✓ Equivalently, the first slot wins $25/40 = 62.5\%$ of all judgements and the second slot only 37.5%: the judge has a +25-point thumb on whatever it reads first.

7 Step 6 — the fix: swap and average

Because the bias is symmetric in the two orders, averaging over both orders cancels it. The **debiased win-rate** for A is its win-rate averaged across the two positions:

$$\text{win}_A^{\text{debiased}} = \frac{1}{2}(\text{win}_{A\text{-first}} + \text{win}_{A\text{-second}}) = \frac{1}{2}(0.70 + 0.45) = 0.575. \quad (3)$$

So A’s *honest* win-rate is 57.5%, not the 70% you would have quoted from a single A-first pass nor the 45% from a single A-second pass. The positional component (± 12.5 points around the true value) has been averaged out; what remains is content. The module states the rule plainly: “always run pairwise judges **twice** with A/B swapped, average, and drop the pair if the two runs disagree.”

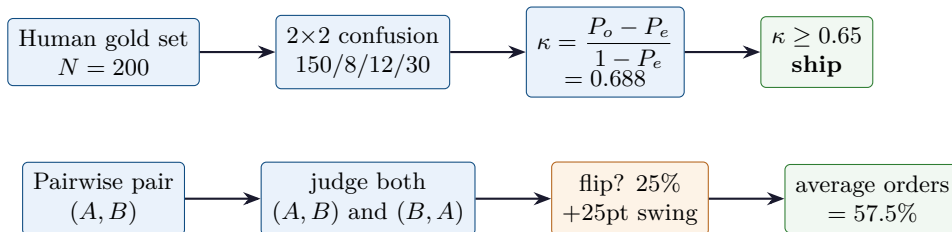
Mini-example — this operation in isolation

Single-order judging is a biased estimator. If you only ever list the challenger first, you bake in a +12.5-point gift to the challenger on every comparison — and your A/B test will “discover” that whichever variant you happened to put first is better. Swap-and-average turns the bias from a systematic error into a wash. The cost is exactly $2\times$ the judge calls; the payoff is a number that is about quality instead of layout.

Intuition

A judge that flips its verdict when you only change the order is not measuring quality — it is measuring *position*. The flip rate is a direct read-out of how much of your pairwise signal is noise: at 25% flips, a quarter of your raw verdicts would reverse under a layout change you would never notice. Swap-and-average is the cheapest honest thing you can do, and a pair that still disagrees after swapping is a pair the judge genuinely cannot call — score it a tie, not a win.

8 The validation pipeline, drawn



Top lane: pointwise judges are gated on κ against humans. Bottom lane: pairwise judges are additionally run in both orders, the flip rate is measured, and the win-rate is taken as the average over orders. Neither number is trustworthy without its lane.

9 Validation cheat-sheet

Observed agreement	$P_o = \frac{n_{11} + n_{22}}{N}$
Chance agreement	$P_e = p_{J,\text{pass}} p_{H,\text{pass}} + p_{J,\text{fail}} p_{H,\text{fail}}$
Cohen's κ	$\kappa = \frac{P_o - P_e}{1 - P_e}$
Deploy floor	ship only if $\kappa \geq 0.65$
Disagreement rate	$1 - P_o$
Position-bias rate	$\frac{\text{\#pairs whose winner flips on swap}}{m}$
Positional swing	$\text{win}_{A\text{-first}} - \text{win}_{A\text{-second}}$
Debiased win-rate	$\frac{1}{2}(\text{win}_{A\text{-first}} + \text{win}_{A\text{-second}})$

10 An honest word about these numbers

The four confusion cells (150, 8, 12, 30) and the 20 pairwise verdicts are illustrative round numbers, chosen so the sums are followable by hand — exactly as the cost-by-hand document used round token sizes. Your real cells depend on your judge, your rubric, and your gold set; a sharper rubric typically buys 5–10 κ points. What is *not* illustrative is the structure: κ always subtracts the base-rate freebie (P_e) from raw agreement and rescales by what is achievable ($1 - P_e$), so a high P_o over a skewed class distribution will always hide a low κ . And the position-bias correction is invariant: a pairwise verdict that changes under order swap is, by definition, partly positional, and the only honest estimate is the average over both orders. Change the constants and the 0.68 and 57.5% move; the two formulas do not. Carry those two — the chance correction and the swap-average — into every judge you ever quote a number from.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the κ computation, the below-floor contrast, and the position-bias swing; change the confusion cells or the pairwise verdicts to validate your own judge.

```

1 # judge_validation.py -- reproduces every number in
2 # "Validating an LLM Judge: Kappa and Position Bias".
3 from math import sqrt
4
5 # ----- Cohen's kappa: judge vs human, N=200 gold trajectories -----
6 n11, n12, n21, n22 = 150, 8, 12, 30 # both-pass, false-pass, false-fail, both-fail
7 N = n11 + n12 + n21 + n22
8
9 Po = (n11 + n22) / N # observed agreement
10 j_pass = (n11 + n12) / N # judge marginals
11 j_fail = (n21 + n22) / N
12 h_pass = (n11 + n21) / N # human marginals
13 h_fail = (n12 + n22) / N
14 Pe = j_pass * h_pass + j_fail * h_fail # chance agreement
15 kappa = (Po - Pe) / (1 - Pe)
16
17 print(f"N={N} Po={Po:.4f} Pe={Pe:.4f} kappa={kappa:.4f} "
18       f"disagree={(1-Po):.4f}")
19

```

```

20 # below-floor judge on the same N for contrast
21 def kfrom(a, b, c, d):
22     n = a + b + c + d
23     po = (a + d) / n
24     pe = ((a+b)/n)*((a+c)/n) + ((c+d)/n)*((b+d)/n)
25     return po, pe, (po - pe) / (1 - pe)
26 po2, pe2, k2 = kfrom(128, 22, 28, 22)
27 print(f"below-floor: Po={po2:.4f} kappa={k2:.4f} disagree={(1-po2):.4f}")
28
29 # ----- Position bias: 20 pairs judged in both orders -----
30 # pass1 winner (A shown first), pass2 winner (B shown first), each in {'A','B'}
31 p1 = ['A']*14 + ['B']*6 # A wins 14/20 when listed first
32 p2 = ['A']*9 + ['B']*11 # A wins 9/20 when listed second
33 pairs = list(zip(p1, p2))
34 m = len(pairs)
35
36 flips = sum(1 for a, b in pairs if a != b) # winner changed on swap
37 A_pos1 = sum(1 for a, b in pairs if a == 'A') # A wins while first
38 A_pos2 = sum(1 for a, b in pairs if b == 'A') # A wins while second
39 swing = (A_pos1 - A_pos2) / m
40 debiased = (A_pos1/m + A_pos2/m) / 2
41 first_slot = sum((a == 'A') + (b == 'B') for a, b in pairs) / (2*m)
42
43 print(f"pairs={m} flips={flips} pos_bias_rate={flips/m:.2f}")
44 print(f"A win-rate pos1={A_pos1/m:.2f} pos2={A_pos2/m:.2f} "
45       f"swing={swing*100:.0f}pts")
46 print(f"first-slot win-rate={first_slot:.3f} debiased_A={debiased:.3f}")
47
48 # Expected output:
49 # N=200 Po=0.9000 Pe=0.6798 kappa=0.6877 disagree=0.1000
50 # below-floor: Po=0.7500 kappa=0.3060 disagree=0.2500
51 # pairs=20 flips=5 pos_bias_rate=0.25
52 # A win-rate pos1=0.70 pos2=0.45 swing=25pts
53 # first-slot win-rate=0.625 debiased_A=0.575

```

— end of document —